



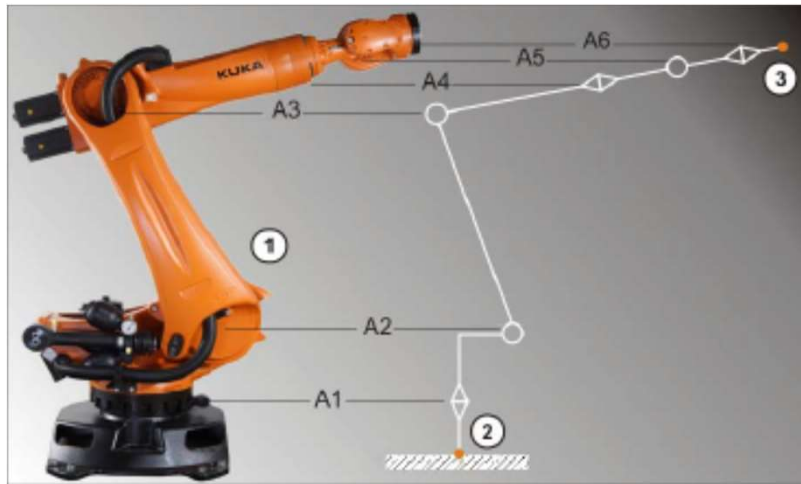


Systèmes de coordonnées et types de déplacements

Sommaire

1. Comment
2. Les différents systèmes coordonnées
3. Transformations cinématiques
4. Les systèmes de coordonnées articulaires et catésiens
5. Repères usuels
6. Différents types de déplacements
7. Mouvement cartésiens relatifs
8. Lissage des trajectoires

Comment bouge un robot?



Hes-so
Haute École Spécialisée
de Suisse romande

Les systèmes de coordonnées

La tâche fondamentale d'un robot est de se positionner dans l'espace, selon une *translation* et une *orientation* donnée. Deux systèmes de coordonnées différents sont utilisés :

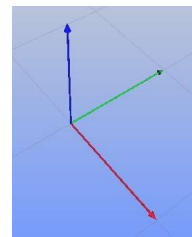
Le système de coordonnées **articulaire** (*joint*), système de coordonnées natif du robot industriel. Les articulations sont de type angulaire (rotation) ou linéaire (translation). Une position articulaire permet de définir la position de chaque axe mécanique.

Les systèmes de coordonnées

Le système de coordonnées **cartésien**. Deux repères cartésiens principaux sont définis par défaut, l'un fixe, à la base du robot (repère **world**), l'autre mobile, à l'extrémité de l'outil du robot (repère **tool**). Celui-ci est fixé sur le *flasque* du robot, le $(n+1)^{\text{ème}}$ segment, situé lui-même après l'axe n .

Remarques :

- le repère cartésien est toujours orthonormé
- X en rouge, Y en vert et Z en bleu !



Transformations cinématiques

Transformation directe:

Articulaire -> Cartésien

Pour une position articulaire donnée, il n'existe qu'une seule position cartésienne !

Transformation indirecte:

Cartésien -> Articulaire

Pour une position cartésienne donnée, il peut exister plusieurs positions articulaires !

7

La calculation *directe* permet de calculer la position dans le système *cartésien* (world) correspondant à une position articulaire donnée. Le résultat correspond à la multiplication des matrices de transfert de chaque axe du robot (selon la convention de Denavit-Hartenberg).

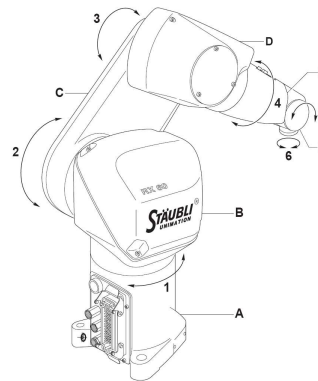
Pour une position articulaire donnée, il n'existe qu'une seule position cartésienne.

La calculation *inverse* permet de calculer la ou les positions articulaires correspondant à une position cartésienne donnée. Les cas où les solutions sont multiples correspondent aux différentes *configurations* possibles du bras (possible pour les architectures série et parallèle).

Pour une position cartésienne donnée, il peut exister plusieurs positions articulaires.

Le système de coordonnées articulaires

$jPosArticulaire = (j1, j2, j3, j4, j5, j6)$



Le système articulaire définit une position mécanique pour chaque axe composant le bras, selon une valeur en [°] pour les articulations angulaires et en [mm] pour les articulations linéaires. Il s'agit d'une valeur brute directement générée par les codeurs. Ces n valeurs, correspondant aux n degrés de liberté du bras, définissent non seulement une position géographique du système de préhension mais également une configuration physique du bras. La combinaison de ces n valeurs définit une position **unique** du bras et une configuration donnée.

Le système de coordonnées cartésien

$$cPosCartésienne = (X, Y, Z, rX, rY, rZ)$$

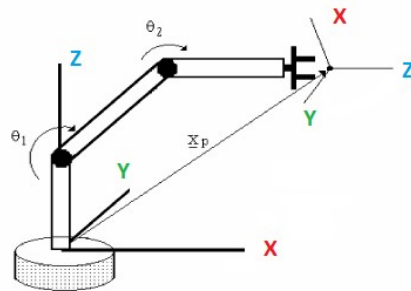


Figure 1. Base Coordinate system.

Le système *articulaire* peut être difficile à visualiser lorsque qu'il faut interpréter des mouvements rectilignes ou des déplacements relatifs en [mm], notamment dans le cas des robots à articulation angulaires. Afin de pouvoir s'affranchir du système *articulaire* et de réaliser aisément les différentes tâches qui leur incombent, les robots industriels possèdent un deuxième système de coordonnées, le système *cartésien*. Celui-ci définit deux repères principaux :

le repère *world*, fixe, à la base du robot. (La position exacte va varier selon le constructeur)

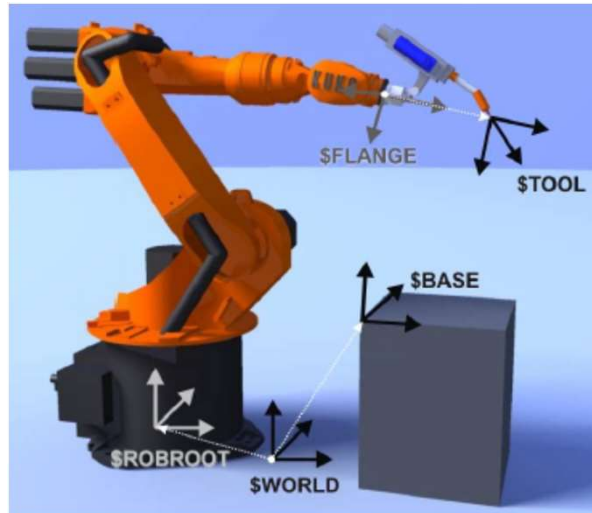
le repère *tool*, mobile, à l'extrémité de l'outil/préhenseur monté sur le robot, celui-ci étant fixé sur le *flasque* du robot, le $(n+1)^{\text{ème}}$ segment, situé lui-même après l'axe n .

Ce système de coordonnées permet de se représenter la position du robot dans l'espace plus simplement qu'avec uniquement les positions angulaires. De plus, on pourra appliquer la même trajectoire à deux robots différents (pour autant bien sûr que les limites physiques respectives de chacun soient respectées). En effet c'est le contrôleur qui va ensuite faire le travail pour calculer les positions angulaires correspondantes. Il s'agit donc également de *généraliser* les trajectoires.

Une position cartésienne est définie par trois translations et trois orientations dans

l'espace. On travaille généralement en [mm] pour les translations et en [°] pour les orientations.

Repères usuels



10

Hes-so
Haute École Spécialisée
du Sud-ouest suisse

WORLD : Repère de base de la cellule, tout les mouvements en un points sont effectués en fonction du repère **WORLD**.

ROBROOT : Position de base du pied du robot en fonction du repère world. Utilisé par le contrôleur pour les transformations cartésiennes. Dans le cas de nos robots Staubli, ce repère est lié avec le repère **WORLD**.

FLANGE : Repère Flasque ou tool_null. Correspond au repère **TOOL** lorsque aucun outil n'est configuré. C'est sur ce repère que vient appliquée la transformation outil.

TOOL : Lorsqu'un outil est configuré, ce repère représente le bout d'outil. Il est donc généralement orienté avec l'axe Z dans le «sens naturel» de l'outil. Lorsqu'un mouvement est effectué, on vient en fait aligner le repère **TOOL (aussi appelé TCP)** sur un repère défini en fonction du world.

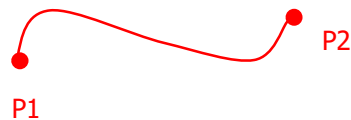
BASE : Représente un repère défini par l'utilisateur. Dans le cas présent, il représente la zone de travail. Il permet, une fois le repère **TOOL** alignée sur ce dernier, d'effectuer des mouvement relatifs alignés sur la zone de travail ou autre. Il est défini en fonction du repère **WORLD**.

Mouvements articulaires synchronisés

Movej:

- Déplacement le + rapide
- Mouvements des moteurs synchronisés
- Trajectoire difficile à prévoir

bool movej(*locj* pos, *bool* break*)



11

Hes-so
Haute École Spécialisée
des Sciences Appliquées

Cette opération permet de positionner le robot en précisant une position *angulaire* ou *articulaire* [°] pour chaque axe. Dans le cas d'un robot avec un axe linéaire, le principe est le même, la position sera cependant cette fois *linéaire* [mm].

Le contrôleur va gérer la vitesse des moteurs afin qu'ils arrivent tous en même temps à destination. Le moteur le plus lent ou ayant le plus de chemin à parcourir définit le temps minimum du mouvement.

La trajectoire de l'outil n'est pas gérée et est difficile à se représenter!

Durant un mouvement articulaire, la *configuration* de départ n'est pas maintenue!

[SYNAPXIS]

pos locj position articulaire

break* bool arrêt au point

Exécute un mouvement vers la position articulaire *pos*. En fonction de la position courante du robot, le mouvement est généré de manière à ce que chaque axe réalise sa

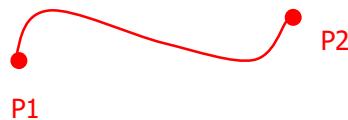
variation de position en un temps identique. Le paramètre *break* indique si un arrêt doit être marqué au point en mode désynchronisé (**setSynchronizedMove**), et vaut *false* par défaut. L'instruction retourne *true* en cas de succès.

Mouvements cartésiens synchronisés

Move:

- Déplacement le + rapide
- Mouvements des moteurs synchronisés
- Trajectoire difficile à prévoir

bool move(*locc* pos, *real* approach*, *bool* break*)



Cette opération permet de positionner le robot en précisant une position *cartésienne* sur laquelle le robot viendra aligner son repère *tool*. Le contrôleur va gérer la vitesse des moteurs afin qu'ils arrivent tous en même temps à destination. Le moteur le plus lent et/ou ayant le plus de chemin à parcourir définit le temps minimum du mouvement.

La trajectoire de l'outil n'est pas gérée et est difficile à anticiper!

Durant un mouvement cartésien, la *configuration* d'arrivée est la même que celle de départ!

[SYNAPXIS]

pos locc position cartésienne

approch* real décalage selon -z (approche)

break* bool arrêt au point

Exécute un mouvement vers la position cartésienne *pos*. En fonction de la position courante du robot, le mouvement est généré de manière à ce que chaque axe réalise sa

variation de position en un temps identique. La configuration de la position finale est identique à celle de la position initiale. Le paramètre *break* indique si un arrêt doit

être

marqué au point en mode désynchronisé (**setSynchronizedMove**), et vaut *false* par défaut. L'instruction retourne *true* en cas de succès.

Mouvement cartésien interpolé (linéaire)

Moves:

- Déplacement le + court
- Interpolation des axes afin de suivre une trajectoire rectiligne

bool moves(*locc* pos, *real* approach*, *bool* break*)



13

Hes-so
Haute École Spécialisée
du Sud de l'Occidentale

Cette opération permet de positionner le robot en précisant une position *cartésienne* sur laquelle le robot viendra aligner son repère *tool*. Le contrôleur va gérer la trajectoire afin que l'outil suive une ligne droite entre le point de départ et d'arrivée.

Durant un mouvement cartésien, la *configuration* d'arrivée est la même que celle de départ!

[SYNAPXIS]

pos locc position cartésienne

approch* real décalage selon -z (approche)

break* bool arrêt au point

Exécute un mouvement vers la position cartésienne *pos*. En fonction de la position courante du robot, le mouvement est généré de manière à ce que l'extrémité du *tool* réalise une translation rectiligne et une rotation sphérique linéaire (SLERP). La configuration de la position finale est identique à celle de la position initiale. Si le paramètre *approch* est spécifié, la position de destination est décalée d'autant selon -z. Par défaut, ce paramètre vaut 0. Le paramètre *break* indique si un arrêt doit être marqué au point en mode désynchronisé (**setSynchronizedMove**), et vaut *false* par

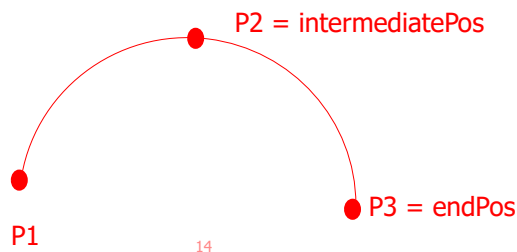
défaut. L'instruction retourne *true* en cas de succès.

Mouvement cartésien interpolé (circulaire)

Movec:

- Interpolation des axes afin de suivre une trajectoire circulaire
- Nécessite un point intermédiaire

bool movec(*locc* intermediatePos, *locc* endPos, *bool* break*)



Hes-so
Haute Ecole Spécialisée
du Sud-ouest suisse

Cette opération permet de positionner le robot en précisant une position *cartésienne* sur laquelle le robot viendra aligner son repère *tool*. Le contrôleur va gérer la trajectoire afin que l'outil suive un arc de cercle entre le point de départ et d'arrivée. Le rayon de l'arc est donné par un point intermédiaire.

Durant un mouvement cartésien, la *configuration* d'arrivée est la même que celle de départ!

Attention! Afin de définir un cercle/arc de cercle, il faut mathématiquement 3 points différents. Il est donc impossible de faire un cercle complet en une seule instruction. En effet, Le point de départ à d'arrivée sont confondus, le cercle ne peut donc pas être calculé sans information supplémentaire tel que le rayon/diamètre et position du centre par exemple.

[SYNAPXIS]

intermediatePos locc position intermédiaire

endPos locc position finale

break* bool arrêt au point

Exécute un mouvement circulaire vers la position cartésienne *endPos*, et passant par *intermediatePos*. En fonction de la position courante du robot (P1), le mouvement

est généré
de manière à ce que l'extrémité du *tool* réalise un arc de cercle. La configuration de
la
position finale est identique à celle de la position initiale. Le paramètre *break*
indique si
un arrêt doit être marqué au point en mode désynchronisé (**setSynchronizedMove**),
et vaut *false* par défaut. L'instruction retourne *true* en cas de succès.

Mouvement cartésiens relatifs

- Pas d'opération native dans Synaxis
- Combinaison de l'addition et de l'opération trans()

locc trans(**real** x, **real** y*, **real** z*, **real** rx*, **real** ry*, **real** rz*)

cPosC = cPosA + cPosB

L'opération d'addition applique en premier la translation (X, Y, Z), puis les rotations (rX, rY, rZ)

-> Attention à l'ordre des opérations!

-> Attention à la direction des translations!

15

Hes-so
Haute Ecole Spécialisée
des Sciences Appliquées

Dans Synaxis, il n'y a pas d'opération native pour effectuer un déplacement relatif. Il est donc nécessaire de combiner plusieurs opérations, notamment l'opération d'addition ainsi que l'opération trans().

L'opération trans() retourne une position cartésienne (locc) en fonction des paramètres d'entrée.

L'opération d'addition s'effectue dans l'ordre des élément des locc.

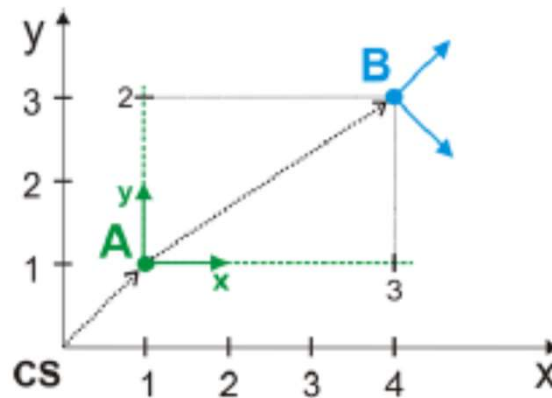
On va donc trouver en premier lieux les trois translation (l'ordre n'influence pas le résultat) puis les trois rotation.

Dans le cas des rotations, il faut être très attentifs à respecter l'ordre rX, puis rY, puis rZ!

On sera également attentif au fait que la translation s'effectue dans les directions correspondante au premier point de l'addition.

Mouvement cartésiens relatifs

$$P_b = P_a + \text{trans}(x, y, z, r_x, r_y, r_z)$$

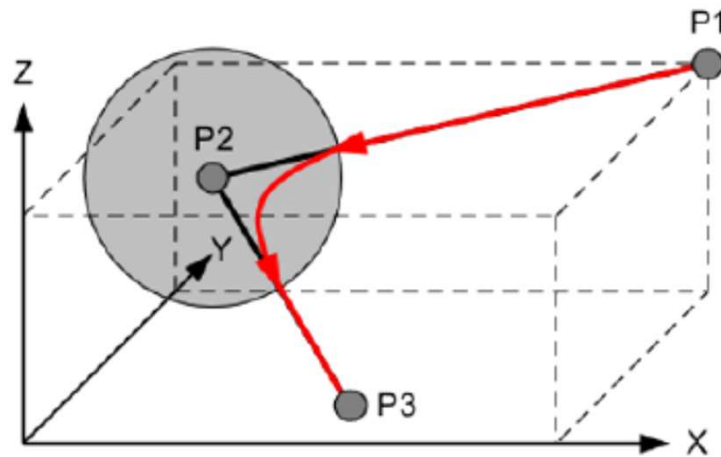


16

Hes-so
Haute École Spécialisée
de Suisse occidentale

La manière d'utiliser ces opération est donc d'additionner au point de départ une translation/rotation définie par l'opération `trans()`.
On effectue ensuite un mouvement **absolu** (tous les mouvements vu plus haut) afin de s'aligner sur le point **relatif** ainsi créé.

Lissage des trajectoires



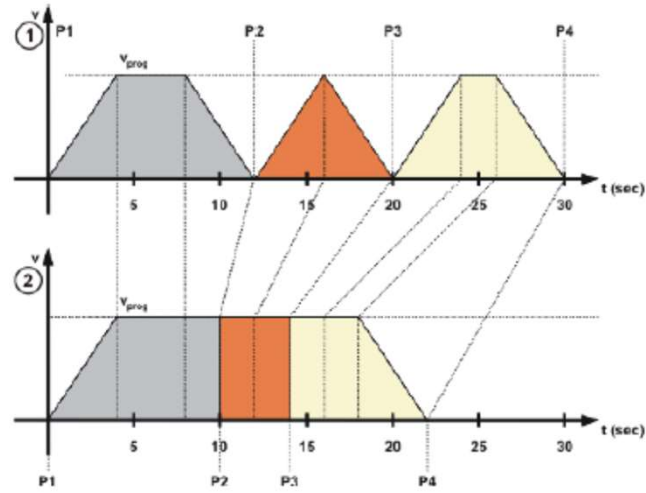
17

Hes-so
Haute École Spécialisée
de Suisse romande

Si rien n'est précisé, le robot va effectuer un arrêt complet à chaque position définie par un move, quel que soit son type.

Lorsque l'on désire suivre une trajectoire avec une vitesse outil constante, on peut utiliser l'option de lissage fournie par Synapxis.

Lissage des trajectoires



Lissage des trajectoires

```
setBlending(5, 10) // Distances ou le robot quitte et rejoint la trajectoire
```

```
setSynchronizedMove(false) // Active le mode désynchronisé
```

```
move(pt1, 0.0, false) //false -> Désactivation de l'arrêt au point
```

```
move(pt2, 0.0, false)
```

```
move(pt3, 0.0, false)
```

```
waitEndMove() // Attente de la fin des mouvements précédents
```

Gestion de la vitesse dans Synpaxis

- **setSpeed(real speed)**
 - Permet de régler la vitesse programme en %
 - Maximum 2000% dans le cas du robot Staubli
- **setSpeedLinear(real speed, real speedInMMPS)**
 - Permet de régler la vitesse d'avance durant le mouvements linéaires
 - Maximum 9999 mm/s dans le cas du robot Staubli
- **setAccel(real accel, real decel*)**
 - Permet de régler l'accélération et la décélération
 - Maximum 1600% dans le cas du robot Staubli